

Hybrid Event-Adaptive Action Representations for Robot Learning

Shaojiang Wang Wenbo Wang Bin Wu*

Institute of AI for Industries, Chinese Academy of Sciences, Nanjing, China

seuhardware@aliyun.com wbwang@iaii.ac.cn

wubin@iaii.ac.cn

Abstract

Robot actions are continuous, temporally structured, and sensitive to small metric errors, while the sequence models used for robot learning benefit from compact and predictable interfaces. We introduce Hybrid Event-Adaptive Action Representations, a framework that discovers variable-duration events from continuous action trajectories and exposes them as a policy-facing intermediate representation. The discrete instantiation, Hybrid Event-Adaptive Tokenization (HEAT), uses a discrete-continuous event code composed of finite scalar quantization (FSQ), a bounded residual, and a learned duration for autoregressive action policies. Continuous-HEAT is the continuous instantiation: it retains continuous event latents and conditions a flow-matching action expert in a Qwen3.5 vision-language-action (VLA) model. Both instantiations use action-derived event supervision during training while avoiding per-sample action leakage at inference. Experiments with HEAT on LIBERO (Benchmarking Knowledge Transfer for Lifelong Robot Learning), MetaWorld, and RoboMimic establish the benefits of event-adaptive timing and hybrid action coding. Experiments with Continuous-HEAT further show that the same event representation remains effective with a Qwen3.5-0.8B backbone in an approximately 1B-parameter overall model.

Keywords: Action representation, event-based control, robot learning, imitation learning, flow matching, action tokenization.

1 Introduction

Robot learning is undergoing a transition from task-specific control pipelines to general sequence models. Transformer-based policies trained on broad robot datasets, language instructions, and visual observations have demonstrated strong potential for generalization across tasks and embodiments [1–3]. In these systems, observations, language, and actions are increasingly treated as elements of a common sequence interface, allowing policies to reuse the modeling machinery that has scaled in language and vision. This interface, however, hides a fundamental asymmetry. Words and image patches are naturally represented as symbols or discrete units, whereas robot actions are continuous, metric, and tightly coupled to physical contact. A small error in end-effector pose, gripper timing, or insertion direction may transform a successful manipulation into a collision, slip, or missed grasp.

The representation of actions is therefore not a peripheral engineering choice; it is a central determinant of what an autoregressive robot policy can learn. Continuous-action policies avoid discretization by directly predicting action chunks. Action Chunking with Transformers (ACT) improves temporal coherence through action chunking [4], and Diffusion Policy models multimodal

*Corresponding author.

HYBRID EVENT-ADAPTIVE ACTION REPRESENTATIONS

A shared action-event interface for robot learning, instantiated as HEAT and Continuous-HEAT

ROBOT LEARNING

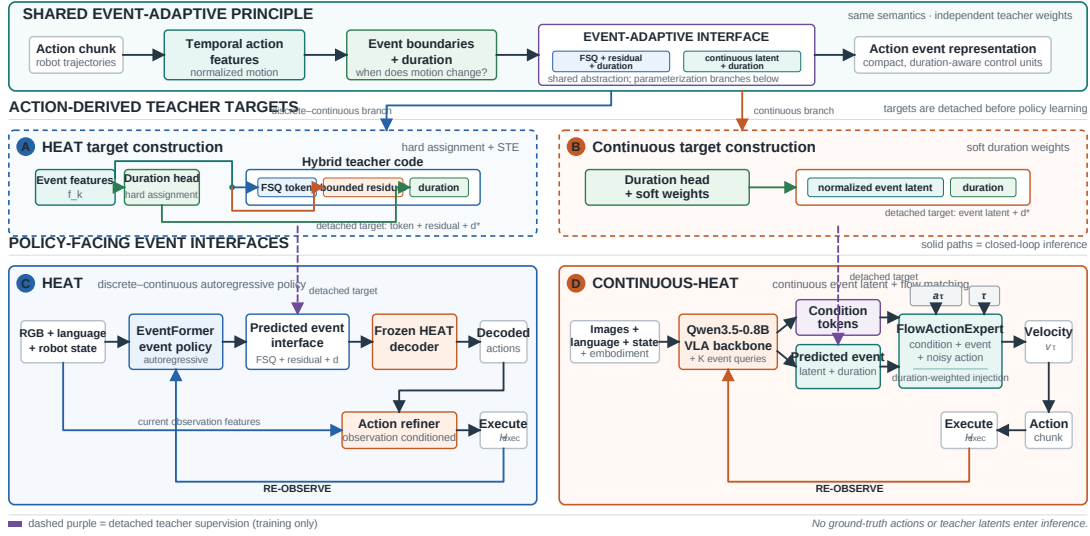


Figure 1: Hybrid event-adaptive action representations. Both instantiations discover variable-duration events. HEAT emits an FSQ token, a bounded residual, and a duration for an autoregressive policy; Continuous-HEAT emits a continuous event latent and a duration to condition a Qwen3.5-0.8B backbone plus flow-matching action expert in an approximately 1B-parameter overall model.

action distributions through iterative denoising [5]. Flow-based policies provide another expressive continuous alternative [6, 7]. These methods preserve metric precision, but their continuous prediction heads do not by themselves provide a compact symbolic interface for next-token prediction. Conversely, autoregressive vision-language-action models commonly discretize actions, for example through per-dimension bins [3], learned latent codes [8, 9], or more specialized action tokenizers such as Frequency-space Action Sequence Tokenization (FAST) and Ordered Action Tokenization (OAT) [10, 11]. Such tokenization enables scalable discrete sequence modeling, yet it also determines the compression rate, the structure of the token space, and the fidelity with which continuous controls can be recovered.

Existing action tokenizers expose a persistent tension among these desiderata. Uniform binning is simple and totally decodable, but the sequence length grows with both action dimension and horizon, and the resulting order is a serialization over coordinates rather than a structure aligned with manipulation phases. Compression-oriented tokenizers reduce sequence length, but a compact discrete code must often carry both global motion intent and fine metric detail. Ordered tokenization further improves compatibility with autoregressive next-token prediction by arranging tokens from coarse to fine [11]; however, ordering alone does not explicitly model how long a manipulation phase should last, nor does it remove the quantization burden placed on discrete codes. In contact-rich robot control, the missing information is frequently not another semantic category, but a small continuous correction or a phase boundary that shifts by a few frames.

This observation motivates an event-centric view of action representation. Manipulation trajectories are rarely uniform streams of equally informative timesteps. A reach may span many frames and admit smooth interpolation; a grasp closure, contact transition, or insertion correction may occupy only a short interval but require high precision; transport and placement may alternate between stable motion and abrupt local adjustment. Treating every frame, every fixed temporal block, or every latent token as having the same temporal status forces a policy to spend capacity on

both redundant smooth motion and sparse critical events. A more natural action interface should represent reusable motion events, allow those events to take variable durations, and still recover the continuous details required for execution.

We argue that an effective action representation for robot learning should satisfy three requirements. First, it should be compact enough for a policy to reason over a short sequence rather than hundreds of per-frame or per-dimension values. Second, it should be temporally adaptive, allocating event boundaries according to the structure of an action chunk rather than a predetermined grid. Third, it should remain metrically faithful and executable in closed loop. These requirements are complementary: temporal adaptivity exposes manipulation phases, while either discrete-continuous or continuous event contents preserve the control detail required by physical interaction.

We propose Hybrid Event-Adaptive Action Representations, shown in Fig. 1. Given an H -step continuous action chunk, the framework learns up to K temporal events whose durations cover the chunk. We study two policy-facing instantiations. HEAT pools each event and emits a discrete-continuous action code:

$$e_k = (z_{\text{tok}}^k, z_{\text{cont}}^k, d_k), \quad (1)$$

where z_{tok}^k is an FSQ discrete token [12], z_{cont}^k is a bounded continuous residual, and d_k is the event duration. In contrast, Continuous-HEAT retains a continuous event latent and its duration, and uses these variables to condition a flow-matching action expert in a Qwen3.5 VLA. In both cases, event content and duration are learned from action trajectories; the difference is whether event content is exposed as an autoregressive token code or as a continuous conditioning signal. For HEAT, a frozen event decoder and EventFormer policy predict event codes autoregressively; the decoded chunk is refined and executed in a short receding-horizon loop. For Continuous-HEAT, Qwen event queries predict continuous event latents and durations in one multimodal forward pass, after which a flow-matching action expert generates the continuous action chunk. Thus the same event abstraction can mediate both symbolic autoregressive prediction and continuous generative control.

This paper makes four contributions:

- We introduce a unified event-adaptive action representation that separates event content from variable temporal extent.
- We present HEAT, a discrete-continuous event representation with FSQ tokens, bounded residuals, and autoregressive decoding.
- We present Continuous-HEAT, a continuous event-conditioned interface for Qwen3.5 and flow-matching action generation.
- We evaluate HEAT on three manipulation benchmarks and demonstrate that Continuous-HEAT preserves the event-adaptive gains at VLA scale.

2 Related Work

2.1 Generative Robot Policies

Modern imitation learning policies increasingly predict action chunks rather than single-step controls. Chunked prediction improves temporal consistency and amortizes inference over multiple low-level actions, and it has become a common design in both compact visuomotor policies and large robot foundation models. ACT uses a transformer-style sequence model with action chunking for fine-grained manipulation [4]. Diffusion Policy models the conditional distribution over continuous action

trajectories through iterative denoising, providing a strong mechanism for multimodal continuous control [5]. Flow-matching policies offer a related continuous generative formulation and have recently been adopted as action experts in large vision-language-action systems [6, 7].

Autoregressive policies take a different route by factorizing action generation into sequential predictions. Their appeal follows from the success of Transformers in language and vision [13]: a single sequence model can condition on observations, language, histories, and previously generated action symbols. Open X-Embodiment, Octo, and OpenVLA demonstrate the promise of this paradigm at larger data and model scales [1–3]. The central challenge is that robot actions must ultimately be executed as continuous commands. Thus, the performance of an autoregressive robot policy depends not only on the policy backbone, but also on the interface that maps between continuous actions and modelable sequences.

2.2 Action Representations and Tokenization

Visual pretraining is a complementary source of transferable structure: A Universal Visual Representation for Robot Manipulation (R3M) and Masked Visual Pre-training for Motor Control (MVP) learn visual representations explicitly for robot manipulation and motor control [14, 15].

Action tokenization defines this interface by mapping continuous action chunks to discrete or partially discrete representations. Per-dimension binning is the simplest approach and is used in several language-conditioned robot policies [3]. It has the advantage of a transparent and well-defined decoder, but its token count grows with both action dimension and horizon, and its serialization over coordinates provides little temporal abstraction. FAST improves compression by transforming action trajectories into a frequency-space representation and applying subword tokenization [10]. OAT studies action tokenization explicitly from the perspective of autoregressive learning, emphasizing compression, decodability, and causal token ordering; it uses register tokens, FSQ, and ordering-inducing training to obtain prefix-decodable action tokens [11, 12, 16].

Learned latent-action methods provide another route to compact action representations. The Vector-Quantized Variational Autoencoder (VQ-VAE) introduced discrete latent bottlenecks for neural representation learning [17]; later robot policies use related quantized latents to represent behavior modes, skills, or action chunks. The Vector-Quantized Behavior Transformer (VQ-BeT) models behavior generation through latent actions [8], while Self-Supervised Skill Abstractions for Learning Continuous Control (QueST) learns reusable skill abstractions [9]. Skill-prior methods further show that latent action abstractions can be learned from offline experience [18]. These methods show that discrete or quantized action representations can capture reusable structure and multimodality. However, when the representation is purely discrete, the same token must account for coarse motion identity and fine metric corrections. HEAT instead keeps a discrete bottleneck for event identity but augments it with a bounded continuous residual, reducing quantization pressure without abandoning the autoregressive token interface.

2.3 Temporal Structure and Closed-loop Execution

Temporal abstraction is especially important for manipulation because action trajectories are not uniformly informative. Reaching, contact formation, grasp closure, transport, and placement can occupy different durations and demand different levels of precision. The options literature formalizes temporally extended actions and learned termination conditions, while Hierarchical Reinforcement Learning with Off-policy correction (HIRO) and latent skill-prior methods study practical hierarchical control from data [18–20]. Fixed-horizon action chunks and fixed-length latent sequences improve efficiency, but they do not explicitly indicate where one manipulation phase

ends and another begins. Ordered tokenizers such as OAT introduce a coarse-to-fine structure over latent tokens [11]; skill-oriented methods such as QueST capture higher-level reusable behaviors [9]. In contrast, the event-adaptive framework makes temporal extent an explicit predicted variable: each event has its own duration, and the HEAT tokenizer learns hard temporal boundaries with differentiable straight-through training.

Closed-loop execution is the other side of temporal structure. Long action chunks can stabilize behavior when predictions are accurate, but executing them open loop can amplify distribution shift after contact, object motion, or perception error. Receding-horizon execution mitigates this issue by repeatedly reconditioning on fresh observations. HEAT follows this principle at the event level. Its autoregressive policy predicts an event sequence as a compact plan, the tokenizer decodes it into continuous actions, and only a short prefix is executed before re-observation. This design positions HEAT between purely continuous chunk generators and fully discrete action-token policies: it uses event tokens for scalable prediction while preserving residual continuous control and frequent feedback.

3 Method

3.1 Unified Event-Adaptive Action Representation

We consider imitation learning from demonstrations containing observations o_t , optional language prompts ℓ_t , robot state s_t , and continuous actions $a_t \in \mathbb{R}^{D_a}$. A standard chunked policy predicts an H -step action sequence $A = (a_1, \dots, a_H)$. Our framework inserts an event-adaptive action representation between the demonstration and the policy. An action encoder discovers K ordered events with content vectors u_k and durations d_k :

$$\tilde{A} \xrightarrow{T_\theta} \mathcal{E} = \{(u_k, d_k)\}_{k=1}^K. \quad (2)$$

The policy-facing event code is an instantiation of u_k . For HEAT, $u_k^{\text{HEAT}} = (z_{\text{tok}}^k, z_{\text{cont}}^k)$ and $e_k^{\text{HEAT}} = (z_{\text{tok}}^k, z_{\text{cont}}^k, d_k)$, where the first component is a discrete event identity and the second is a bounded metric residual. For Continuous-HEAT, $e_k^{\text{C-HEAT}} = (z_{\text{event}}^k, d_k)$, where z_{event}^k is a normalized continuous latent. The two instantiations share event timing and action-derived supervision, but use different policy interfaces and decoders.

3.2 Event-Adaptive Temporal Structure

The action encoder first normalizes each action chunk using the dataset action normalizer. A Transformer maps $\tilde{A}$ to temporal features $h_{1:H}$. A duration predictor pools the sequence, converts its logits to simplex weights, adds an implementation-specific nonnegative offset, and renormalizes the resulting lengths to the action horizon:

$$p = \text{softmax}(g(\text{mean}(h_{1:H}))), \quad d_k^{\text{soft}} = H \frac{p_k + \delta}{\sum_{j=1}^K (p_j + \delta)} = H \frac{p_k + \delta}{1 + K\delta}. \quad (3)$$

Here $\delta \geq 0$ controls the soft-duration floor before horizon normalization, and $\sum_{k=1}^K d_k^{\text{soft}} = H$. The HEAT experiments use the minimum-free setting $\delta = 0$ before converting the positive soft durations to integer frame assignments. Continuous-HEAT retains soft assignments and uses $\delta = 0.05$, matching the continuous event encoder implementation. The two variants therefore share the normalized duration parameterization while differing in the floor and in whether the temporal

partition is consumed as a hard or soft assignment. For the uniform-duration ablation, Eq. (3) is replaced with $d_k^{\text{soft}} = H/K$.

The soft durations define cumulative boundaries $b_k = \sum_{j=1}^k d_j^{\text{soft}}$. For an integer time index $t \in \{0, \dots, H-1\}$, the hard event assignment is

$$s_t = \min \left(\sum_{k=1}^K \mathbb{I}[t \geq b_k], K-1 \right). \quad (4)$$

This gives a hard mask $M_{\text{hard}} \in \{0, 1\}^{H \times K}$. To train the duration predictor through discrete boundaries, the event encoder also constructs a soft mask from squared distances to event centers $c_k = b_k - d_k^{\text{soft}}/2$:

$$M_{\text{soft}}(t, k) = \text{softmax}_k \left(-\frac{(t + \frac{1}{2} - c_k)^2}{\tau} \right), \quad (5)$$

where $t + \frac{1}{2}$ denotes the center of frame t and τ is the event-assignment temperature, set to 2.0 by default. The mask used for event pooling is straight-through:

$$M_{\text{STE}} = M_{\text{hard}} + M_{\text{soft}} - \text{stopgrad}(M_{\text{soft}}). \quad (6)$$

The forward pass therefore uses hard temporal segments, while gradients flow through the soft assignment. Event features are normalized weighted averages,

$$f_k = \frac{\sum_{t=0}^{H-1} M_{\text{STE}}(t, k) h_t}{\sum_{t=0}^{H-1} M_{\text{STE}}(t, k) + \epsilon}. \quad (7)$$

For HEAT, the representation exports integer frame counts induced by Eq. (4), i.e., $\bar{d}_k = \sum_t \mathbb{I}[s_t = k]$. Continuous-HEAT instead retains the normalized soft durations and event-weighted pooled features. In both cases, the action-derived event representation is used as a training target for the observation-conditioned policy.

3.3 HEAT: Discrete-Continuous Event Representation

In the HEAT instantiation, each event feature f_k is encoded by a discrete branch and a continuous branch. The discrete branch linearly projects f_k into the FSQ latent dimension and applies the quantizer described above [12]. With the default levels $[8, 5, 5, 5]$, the vocabulary size is $8 \cdot 5 \cdot 5 \cdot 5 = 1000$. The continuous branch predicts a bounded residual

$$z_{\text{cont}}^k = 0.5 \tanh(W_c f_k), \quad (8)$$

so every residual dimension lies in $[-0.5, 0.5]$. During tokenizer training, the implementation applies residual dropout with probability 0.2 and rescales the retained residuals, discouraging the continuous branch from replacing the discrete token.

Decoding is deterministic given tokens, residuals, and durations. The token index is first converted back to its FSQ code. A coarse decoder embeds the FSQ code, a fine decoder embeds z_{cont}^k , and the two embeddings are summed to form an event embedding. The decoder reconstructs frame assignments from the provided durations, expands each event embedding to its assigned frames, adds an event-local positional embedding, and applies a Transformer encoder followed by a linear action head. During tokenizer training, the reconstruction is compared in normalized action space; during policy rollout, the decoded trajectory is unnormalized to obtain executable actions.

3.4 HEAT Policy Interface and Closed-loop Execution

After HEAT representation training, the event encoder and decoder are frozen. The policy receives fused condition tokens from red-green-blue (RGB) observations, robot state, and optional language. In the implementation, RGB streams use a robomimic visual encoder with a ResNet18 backbone and spatial softmax pooling [21]; prompts are encoded with a frozen Contrastive Language-Image Pre-training (CLIP) text encoder when available [22]; low-dimensional state is projected linearly.

The policy predicts the event sequence autoregressively. With teacher forcing, the input at event position k contains a beginning-of-sequence token for $k = 1$, otherwise the previous token, together with the previous residual and previous duration. The event model outputs token logits, a bounded residual using Eq. (8), and durations with the same normalization form as Eq. (3). The codebase supports several sequence backbones. The EventFormer variant used in the main experiments is a causal decoder that cross-attends to condition tokens and optionally adds duration-aware self-attention bias; this backbone is an implementation choice rather than the core representation.

At inference, the policy rolls out K events, updating the next input position with the token, residual, and duration predicted at the current position. The frozen event decoder reconstructs the full event sequence into an H -step trajectory. A lightweight action refiner then cross-attends the first few decoded actions to the current condition tokens and predicts a small bounded correction, with default scale 0.05. Only the first H_{exec} refined actions are returned, where the implementation clamps $H_{\text{exec}} \leq 4$, and the policy re-observes the environment before predicting again. Thus HEAT uses a full event sequence as a compact plan, but executes it through short-horizon closed-loop control.

3.5 Continuous-HEAT: Event-Conditioned Flow-Matching VLA

Continuous-HEAT instantiates the same event-adaptive principle without quantizing event content. A Qwen3.5-0.8B vision-language backbone [23] receives images, language, embodiment information, and state-conditioned prompts, together with K learned event queries. The backbone is part of an approximately 1B-parameter overall model after adding the flow-matching action expert. The query features are projected to normalized continuous event latents z_{event}^k and positive durations d_k . During training, the action encoder supplies privileged targets $(z_{\text{event}}^{k,*}, d_k^*)$ obtained from the ground-truth action chunk. These targets provide event-alignment and duration losses, but they are detached from the policy path and are never available to the action expert at inference.

The predicted event latents and durations condition a Diffusion Transformer (DiT)-style FlowActionExpert that attends jointly to visual-language condition tokens, event tokens, and noisy action tokens. Given a flow time τ , the expert predicts the velocity field for the interpolated action trajectory. At inference, a small number of Euler integration steps generate a continuous action chunk from the Qwen condition and predicted events. Only the first few actions are executed before the environment is observed again. Continuous-HEAT is therefore a continuous event-conditioned policy, rather than an autoregressive discrete tokenizer.

4 Training Objective

4.1 HEAT Training

The tokenizer is trained as an action autoencoder. Given ground-truth continuous actions A , the tokenizer reconstructs $\hat{A}$ after normalization. The primary loss is mean squared reconstruction error,

with an additional penalty on the continuous residual magnitude:

$$\mathcal{L}_{\text{tok}} = \|\hat{A} - \tilde{A}\|_2^2 + \lambda_c \frac{1}{K} \sum_{k=1}^K \|z_{\text{cont}}^k\|_2^2. \quad (9)$$

The implementation uses $\lambda_c = 0.05$. This objective encourages accurate reconstruction while limiting the residual branch.

4.2 Policy Training

The policy is trained after freezing the tokenizer. For each training action chunk, the tokenizer provides teacher event codes $(z_{\text{tok}}, z_{\text{cont}}, d)$. EventFormer is optimized with token cross entropy, residual regression, duration regression, decoded-action losses, and auxiliary event-state losses. Here cross-entropy (CE) denotes the token classification loss:

$$\begin{aligned} \mathcal{L}_{\text{policy}} = & \lambda_z \mathcal{L}_{\text{CE}}(z_{\text{tok}}, z_{\text{tok}}) + \lambda_c \|z_{\text{cont}} - \hat{z}_{\text{cont}}\|_1 + \lambda_d \|\hat{d} - d\|_1 \\ & + \lambda_a \mathcal{L}_{\text{act}} \\ & + \lambda_p (\mathcal{L}_{\text{prog}} + \mathcal{L}_{\text{term}}) \\ & + \lambda_s \mathcal{L}_{\text{state}} \\ & + \lambda_v \mathcal{L}_{\text{visual}}. \end{aligned} \quad (10)$$

The decoded-action term supervises both teacher-token decoding and model-token decoding after the action refiner. The default weights in the training script are $\lambda_z = 1$, $\lambda_c = 10$, $\lambda_d = 1$, $\lambda_a = 25$, $\lambda_p = 0.5$, $\lambda_s = 0.5$, and $\lambda_v = 0.02$.

4.3 Continuous-HEAT Training

Continuous-HEAT is trained with a joint flow-matching and event-alignment objective:

$$\begin{aligned} \mathcal{L}_{\text{C-HEAT}} = & \lambda_f \mathcal{L}_{\text{flow}} + \lambda_e \mathcal{L}_{\text{event}} + \lambda_d \mathcal{L}_{\text{duration}} \\ & + \lambda_r \mathcal{L}_{\text{reconstruction}} + \lambda_{\text{reg}} \mathcal{L}_{\text{latent-reg}} + \lambda_{\text{vl}} \mathcal{L}_{\text{VL}}. \end{aligned} \quad (11)$$

Here $\mathcal{L}_{\text{flow}}$ is the masked flow-matching loss of the action expert. The event loss combines Smooth- L_1 and cosine alignment between Qwen-predicted latents and action-derived teacher latents, while the duration loss aligns their normalized temporal extents. The action-derived encoder reconstruction and latent regularization terms are auxiliary losses that keep the event target identifiable. The optional vision-language (VL) loss is applied only to samples with assistant responses. During inference, the flow expert receives only Qwen-predicted event latents and durations; the action-derived encoder is a privileged training teacher and cannot leak the target action chunk.

The implementation follows the Qwen-VLA pipeline [24]: text-to-action DiT pretraining (T2A), continued pretraining (CPT), and a supervised fine-tuning (SFT) checkpoint used for the matched evaluation. T2A trains the flow action decoder as a language-conditioned action decompressor before visual input is introduced. CPT continues training with visual-language-action data to ground this action prior in observations, while the event-query pathway is enabled within the reported CPT experiment. SFT denotes the separately reported supervised checkpoint; both checkpoints are reported directly in Table 2.

5 Experiments

5.1 Experimental Setup

The HEAT benchmark experiments use zarr-format demonstration datasets with action chunks and synchronized observations. All three benchmark families are converted into the same policy-facing schema: each timestep contains a continuous action, third-person RGB, wrist RGB, end-effector position and orientation, gripper state, joint position, a task identifier, and a language prompt when available. This common representation lets the tokenizer and EventFormer policy use the same action-horizon, observation, and conditioning interface across datasets, while preserving the visual and state information needed for closed-loop manipulation.

The main benchmarks are LIBERO [25], MetaWorld [26], and RoboMimic [21]. The training sets contain 138,090 samples from 10 LIBERO tasks, 1,000,000 samples from 4 MetaWorld tasks, and 60,674 samples from 3 RoboMimic tasks. LIBERO evaluates language-conditioned tabletop rearrangement and appliance interaction, MetaWorld focuses on compact simulated manipulation skills, and RoboMimic provides robosuite-based imitation trajectories. RoboMimic emphasizes demonstration-driven object manipulation, including grasping, lifting, transport, and placement under contact-rich dynamics. Fig. 2 shows two representative samples from each benchmark family: LIBERO Mug/Pudding and Mug/Microwave, MetaWorld Disassemble and Coffee Pull, and RoboMimic Lift and Can. In every panel, the upper image is the third-person camera and the lower image is the wrist camera. The repository also contains RoboCasa [27] conversion and evaluation support built on robosuite [28]. Policies are evaluated by rollout success rate in simulation; unless otherwise noted, the tables report the project result sheet, and available logs record aggregate rollout success without always including per-task LIBERO breakdowns.

For the broad-data VLA pipeline, we additionally inventory 16 converted LeRobot v3 directories spanning CALVIN, DROID, LIBERO, MetaWorld, seven Open X-Embodiment (OXE) sources, RoboMME, RoboMimic, RoboCasa, and Piper. These sources are representative of established language-conditioned and in-the-wild manipulation benchmarks, including CALVIN, DROID, and BridgeData V2 [29–31]. Counted as stored, without cross-directory deduplication, this corpus contains 257,875 episodes and 37,989,863 frames. Its readable files have a summed logical size of at least 185.25 GB; the RoboCasa `extras` directory could not be enumerated and is excluded from this storage figure. The sources are heterogeneous: action width ranges from 4 to 14, the number of cameras from 1 to 4, and collection frequency from 5 to 80 Hz. Fig. 3 summarizes the resulting long-tailed distribution before sampler reweighting. OXE contributes 18.62 million frames (49.0%) and RoboCasa contributes 14.96 million (39.4%); together they account for 88.4% of stored frames. The mean episode length is 147.3 frames, but source-level means range from 32.4 frames for OXE Fractal to 774.9 for OXE FurnitureBench.

All experiments are run on Ubuntu 22.04 with Compute Unified Device Architecture (CUDA) 13.1, PyTorch 2.8, and Python 3.10. Training uses NVIDIA H200 graphics processing units (GPUs) with 141 GB of memory per GPU. The pretraining portion of the approximately 1B-parameter overall model takes approximately 96 GPUs for 60 hours (about 5,760 GPU-hours). The T2A stage uses a global batch size of 96×512 and is trained for one epoch, while the CPT stage uses a global batch size of 96×48 and is trained for 30 epochs.

5.2 Baselines

For the HEAT experiments, we compare against continuous-action imitation baselines and action-tokenization baselines. Continuous baselines include Diffusion Policy [5], ACT [4], and a flow-matching policy [6]. Tokenization baselines include uniform binning, FAST [10], QueST [9], OAT [11],

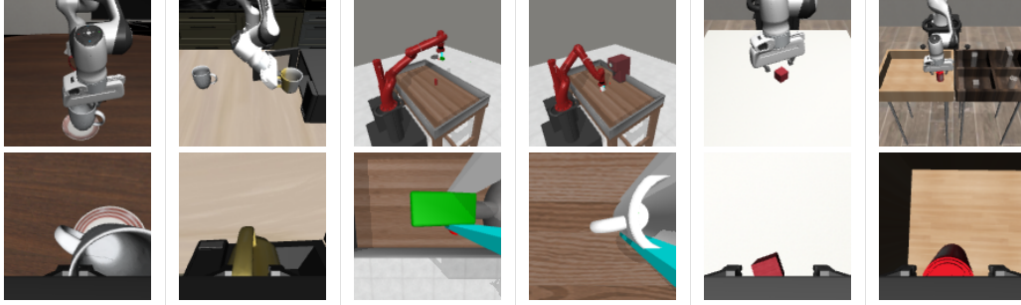


Figure 2: Representative samples from the converted zarr datasets. From left to right: LIBERO Mug/Pudding, LIBERO Mug/Microwave, MetaWorld Disassemble, MetaWorld Coffee Pull, RoboMimic Lift, and RoboMimic Can. The top row shows the third-person camera, and the bottom row shows the wrist camera.

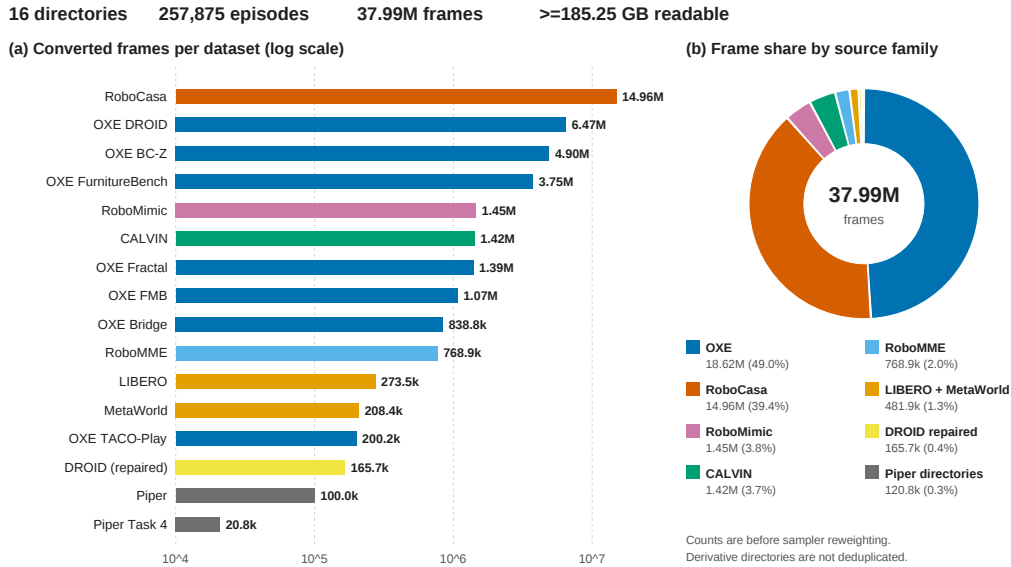


Figure 3: Composition of the 16 converted LeRobot v3 dataset directories. (a) Converted frame counts for each directory on a logarithmic scale. (b) Share of the 37.99 million stored frames after grouping directories by source family. Counts precede sampler reweighting and are not deduplicated across derivative directories.

and VQ-BeT-style vector-quantized action models [8]. The main HEAT row uses the EventFormer policy. For the Continuous-HEAT experiments, the two matched runs are labeled CPT and SFT; the primary controlled baseline is the same Qwen3.5-0.8B backbone and flow-matching action expert with event conditioning disabled.

5.3 Main Results

Table 1 reports LIBERO results. HEAT obtains the best average success rate among the listed methods, with especially strong performance on long-horizon object rearrangement tasks such as Book/Caddy and Mug/Micro.

Table 1: LIBERO success rate (%). The average is over the ten listed LIBERO tasks.

Method	Soup/Sauce	Cheese/Butter	Soup/Cheese	Two Moka	Stove/Moka	Bowl/Drawer	Mugs/Plates	Book/Caddy	Mug/Pudding	Mug/Micro	Avg.
Diffusion	0.0	12.0	0.0	0.0	20.0	24.0	12.0	14.0	2.0	6.0	9.0
ACT	0.0	0.0	0.0	0.0	56.0	0.0	0.0	0.0	4.0	48.0	10.8
Flow	2.0	20.0	6.0	0.0	76.0	16.0	4.0	4.0	0.0	42.0	17.0
Bin (OpenVLA)	1.4	3.2	4.2	9.8	26.6	44.4	3.8	31.8	7.2	14.0	14.6
FAST (π_0)	12.5	6.2	1.2	35.6	52.0	48.2	14.0	25.4	21.0	3.4	22.0
QueST	24.4	14.2	30.4	45.6	79.8	88.2	21.8	66.6	34.6	74.4	48.0
OAT8	24.0	33.8	50.6	60.6	86.8	90.4	31.0	70.4	32.2	77.6	55.7
HEAT (EventFormer)	28.2	36.1	60.5	84.7	80.8	84.0	32.7	100.0	28.9	87.1	62.3

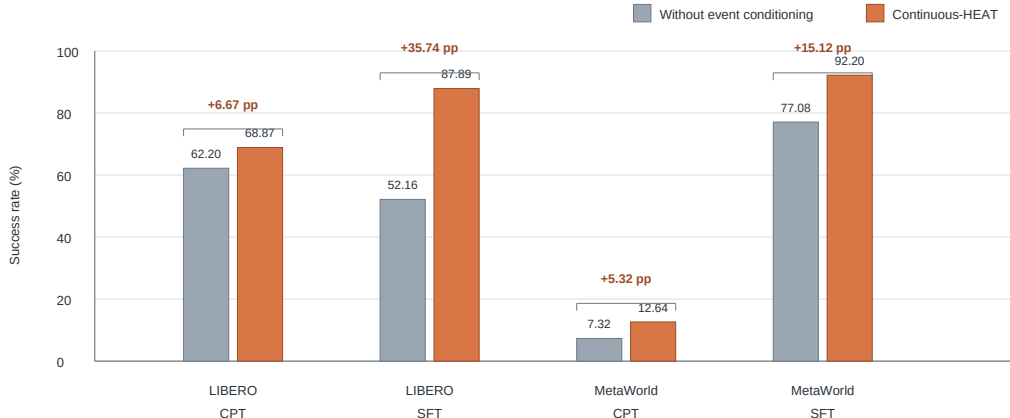


Figure 4: Matched Continuous-HEAT rollout results with the Qwen3.5-0.8B backbone and flow-matching action expert. Bars report success rate over all evaluated episodes; annotations give the absolute event-conditioning gain in percentage points (pp). The four matched pairs are LIBERO/CPT (62.20% vs. 68.87%), LIBERO/SFT (52.16% vs. 87.89%), MetaWorld/CPT (7.32% vs. 12.64%), and MetaWorld/SFT (77.08% vs. 92.20%), where each pair is Without event versus Continuous-HEAT.

Tables 3 and 4 summarize MetaWorld and RoboMimic. HEAT achieves the highest average success rate on RoboMimic and improves substantially over binning, FAST, QueST, and OAT on MetaWorld. VQ-BeT remains a strong MetaWorld baseline, which suggests that event-adaptive timing and latent-action modeling are complementary directions.

5.4 Continuous-HEAT Scalability Study

We additionally evaluate whether event-adaptive action representations remain useful with a Qwen3.5-0.8B vision-language backbone in an approximately 1B-parameter overall model with a continuous action generator. Continuous-HEAT uses Qwen3.5-0.8B event queries and a flow-matching action expert, as described in Section 3. The two reported runs are labeled CPT and SFT throughout this study. For each setting, the backbone, data, optimizer, flow integration steps, and execution horizon are fixed; only the event pathway is toggled. We therefore treat the no-event model as a matched control rather than as a separate baseline family. The evaluation logs contain complete rollout outcomes, so this study reports success counts and rates directly.

Fig. 4 and Table 2 show that event conditioning improves all four matched comparisons. On LIBERO, the gain is +6.67 percentage points (pp) after CPT and +35.74 pp after SFT; on MetaWorld, the corresponding gains are +5.32 and +15.12 pp. Averaged across the four matched settings, Continuous-HEAT improves success by 15.71 pp, so the no-event control is lower by approximately 15 pp on average. The consistent direction across the matched settings supports the

Table 2: Matched Continuous-HEAT rollout results. Each entry is successes/episodes followed by success rate (%).

Suite / checkpoint	Subset	Without event	Continuous-HEAT	Gain (pp)
LIBERO / CPT	All	1053/1693 (62.20)	1166/1693 (68.87)	+6.67
LIBERO / SFT	All	883/1693 (52.16)	1488/1693 (87.89)	+35.74
MetaWorld / CPT	All	183/2500 (7.32)	316/2500 (12.64)	+5.32
MetaWorld / SFT	Overall	1927/2500 (77.08)	2305/2500 (92.20)	+15.12

claim that the event representation transfers to VLA-scale continuous control.

These matched rollout evaluations report absolute gains without significance claims. They should not be read as a direct ranking against the EventFormer results in the preceding tables, because HEAT and Continuous-HEAT use different policy backbones and action decoders.

We further perform a boundary-consistency diagnostic analogous to Fig. 5 on 96 complete episode-start chunks from the converged CPT checkpoint, with 32 samples from each of three evaluation datasets. The action-derived event encoder is used only offline to provide a reference; Qwen receives only the synchronized images and embodiment-aware instruction. Fig. 6 shows one qualitative sample: the solid blue teacher boundaries and dashed orange Qwen boundaries nearly coincide, while the gray dotted uniform boundaries ignore the non-uniform timing of the action changes. The lower traces show the action-change norm and gripper signal, illustrating that learned boundaries allocate temporal resolution around transitions and use longer intervals during relatively stable motion. Across all three datasets, the predicted event durations differ from the reference by 0.028 frames on average, and the mean internal-boundary error is 0.023 frames, compared with 0.585 frames for a uniform eight-event partition. Thus the Qwen event queries reduce boundary discrepancy relative to the uniform reference by 96.1%, while reaching an average event-latent cosine similarity of 0.983. This result measures consistency with the learned action teacher rather than semantic-boundary accuracy, but it verifies that the event interface survives the transition from action-conditioned encoding during training to image-conditioned prediction at inference.

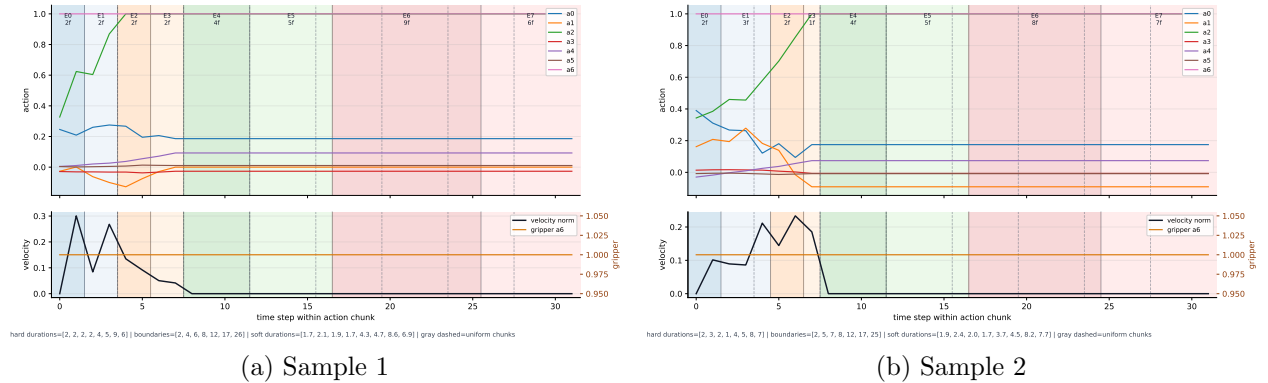


Figure 5: Each subfigure shows one action sample over time. In the upper panel, colored curves are the individual action dimensions, and the colored background bands are the event segments produced by the frozen HEAT tokenizer; the text on each band gives the event index and its duration in frames. In the lower panel, the black curve is the action-velocity norm, which highlights where the action changes sharply, and the orange curve is the gripper action. Gray dashed vertical lines show a uniform temporal partition for comparison. The key observation is that HEAT does not simply split the horizon into equal blocks: it places several short events near high-velocity transition regions and uses longer events when the action becomes stable, indicating event-adaptive temporal segmentation.

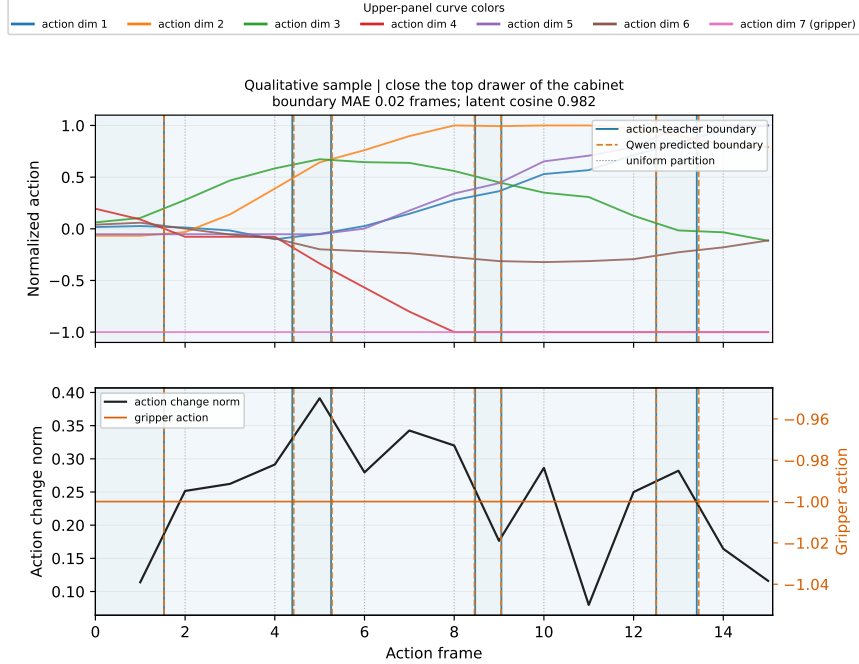


Figure 6: A qualitative event-boundary sample for Continuous-HEAT. The upper plot shows normalized action dimensions; the legend identifies each dimension by color, with the seventh dimension denoting the gripper. Blue solid lines and pale blue bands mark boundaries from the offline action-derived teacher, orange dashed lines mark boundaries predicted by Qwen event queries from images and the embodiment-aware instruction, and gray dotted lines show the uniform partition. In the lower plot, black denotes the action-change norm and orange denotes the gripper action. The close agreement between the two learned partitions, together with their departure from uniform timing, qualitatively indicates that the event interface transfers to image-conditioned Qwen prediction.

5.5 HEAT Representation Reconstruction Analysis

We further evaluate the frozen HEAT representation as an offline autoencoder, independent of policy prediction and simulator rollouts. HEAT encodes a ground-truth validation action chunk from the zarr datasets and decodes it back to continuous actions. We report normalized action mean squared error (MSE), raw action MSE after unnormalization, endpoint error from accumulated end-effector displacement, gripper-dimension MSE, and raw MSE on the first $H_{\text{exec}} = 4$ actions used by short-horizon execution. The values are equally averaged over LIBERO/libero10_N500, four MetaWorld tasks, and three RoboMimic tasks. Table 5 shortens these headers to Norm., Raw, End., Grip., and Exec., respectively. Continuous-HEAT does not use this discrete decoder and is evaluated separately in Table 2.

Table 5 and Fig. 7 show that reconstruction error alone is not a monotonic predictor of rollout success. Coordinate binning is nearly lossless because it allocates symbols at the per-coordinate level, and FAST provides a strong transform-based trajectory compressor. HEAT is instead designed as a compact policy-facing event representation. Among the learned compact latent-action tokenizers in this comparison, HEAT gives the lowest average normalized MSE, raw MSE, endpoint error, gripper MSE, and first-execution raw MSE. This supports that the event-adaptive hybrid code preserves

Table 3: MetaWorld success rate (%).

Method	Box Close	Coffee Pull	Disassemble	Stick Pull	Avg.
Bin	20.2	50.8	77.2	17.0	41.3
FAST	22.8	28.6	85.2	8.8	36.4
QueST	13.6	26.4	23.8	6.0	17.5
OAT	36.8	23.8	72.0	15.0	36.9
VQ-BeT	61.0	44.6	86.6	43.0	58.8
HEAT	76.8	46.0	89.6	43.2	63.9

Table 4: RoboMimic success rate (%).

Method	Lift	Square	Can	Avg.
Bin	51.0	0.0	0.6	17.2
FAST	53.6	0.4	18.0	24.0
QueST	45.2	0.6	40.0	28.6
OAT	60.0	1.0	44.2	35.1
VQ-BeT	69.4	1.0	57.0	42.5
HEAT	94.4	0.0	79.8	58.1

metric action information better than purely discrete latent alternatives, while the main rollout tables indicate that downstream success also depends on autoregressive predictability, temporal structure, and short-horizon closed-loop execution.

5.6 Residual Magnitude Analysis

A potential failure mode of a hybrid tokenizer is that the continuous residual could become a hidden continuous-action bypass, leaving the discrete token to carry little useful information. We therefore analyze the frozen HEAT tokenizer over all action-window chunks in the three benchmark families. For each event, we measure the elementwise residual magnitude $|z_{\text{cont}}|$ and the relative decoder contribution

$$r_k = \frac{\|\text{Dec}_{\text{fine}}(z_{\text{cont}}^k)\|_2}{\|\text{Dec}_{\text{coarse}}(z_{\text{tok}}^k)\|_2}. \quad (12)$$

The first statistic tests whether the bounded residual saturates its allowed interval, while the second tests whether the fine decoder embedding dominates the coarse token embedding. We also report the fine-share statistic $\|\text{Dec}_{\text{fine}}\|_2 / (\|\text{Dec}_{\text{coarse}}\|_2 + \|\text{Dec}_{\text{fine}}\|_2)$. Table 6 uses compact column labels: Res50 and Res95 are the elementwise $|z_{\text{cont}}|$ percentiles, Ratio50 and Ratio95 are the corresponding r percentiles, Fine< 0.5 is the percentage of events whose fine norm is less than half of the coarse norm, and Fine share is the mean fine share.

Table 6 and Fig. 8 show that the residual branch is active but subordinate to the discrete branch. The elementwise residuals do not collapse to zero, which is desirable because they preserve local metric detail. At the same time, the fine-to-coarse ratio remains far below one: the median ratio is 0.33–0.39, the 95th percentile is 0.46–0.53, and no event in the full-dataset logs has $r > 1.0$. The mean fine share is also only 0.26–0.29. RoboMimic has the largest residual magnitudes and contribution ratios, consistent with its contact-rich manipulation trajectories, but even there the residual remains a smaller decoder component than the coarse FSQ token. These statistics support the intended interpretation of z_{cont} : it supplies fine metric correction on top of a discrete event identity, rather than secretly turning the tokenizer into a continuous latent-action model.

Table 5: Offline tokenizer reconstruction error. Lower is better.

Method	Interface	Norm.	Raw	End.	Grip.	Exec.
Bin	Coordinate bins	5.75e-6	1.52e-5	0.15	1.09e-5	1.51e-5
FAST	Transform tokens	2.80e-4	6.47e-4	0.53	2.35e-4	7.42e-4
HEAT	Event hybrid	0.0014	0.0014	0.73	2.62e-4	0.0018
QueST	Skill latent	0.0040	0.0026	1.11	0.0014	0.0032
OAT	Ordered latent	0.0055	0.0067	1.75	0.0047	0.0087
VQ-BeT	VQ latent	0.033	0.087	5.92	0.10	0.10

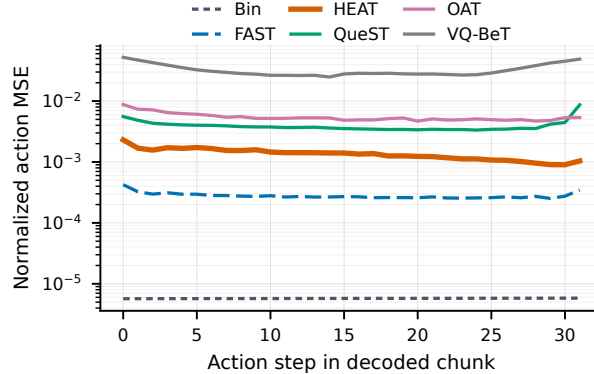


Figure 7: Mean per-step normalized reconstruction MSE across the same validation subsets as Table 5. The log scale shows both near-lossless reconstruction references and compact learned tokenizers.

5.7 Dataset-Level Duration Distribution

To verify that event-adaptive timing is learned across the datasets rather than only appearing in selected examples, we analyze the hard durations exported by the frozen HEAT tokenizer for all action-window chunks in the three benchmark families. Each action chunk has horizon $H = 32$ and $K = 8$ events, so a uniform partition would assign four frames to every event and would have normalized duration entropy 1.0 with zero deviation from uniform. These statistics use the HEAT setting $\delta = 0$ in Eq. (3) and the integer frame counts induced by Eq. (4). Softmax makes every soft duration strictly positive, but it does not impose a one-frame lower bound in this setting. If a soft interval is shorter than one frame, two adjacent continuous boundaries can fall between the same pair of integer sampling locations, so the corresponding hard event receives zero assigned frames. The zero in the LIBERO hard-duration range therefore describes the discrete export rather than a zero-valued soft duration; it does not apply to the soft-duration interface used by Continuous-HEAT. Table 7 uses Range for hard-duration support, Std. for duration standard deviation, MAD for mean absolute deviation from the uniform four-frame allocation, and Entropy for normalized duration entropy. The learned tokenizer consistently departs from the fixed grid: the mean duration remains four frames because the hard assignments partition the horizon, but the standard deviation, duration range, and mean absolute deviation from the uniform allocation are all nonzero. The effect is strongest on MetaWorld, where the tokenizer frequently assigns short intermediate events and a long final event, producing a mean normalized duration entropy of 0.828 and a mean absolute deviation of 2.67 frames from the uniform reference.

Fig. 9 provides a representative view of this dataset-level behavior. On MetaWorld, events

Table 6: Residual magnitude and decoder-contribution statistics.

Family	Chunks	Res50	Res95	Ratio50	Ratio95	Fine< 0.5	Fine share
LIBERO	138,090	0.087	0.257	0.336	0.459	98.6	0.257
MetaWorld	1,000,000	0.097	0.296	0.332	0.456	99.7	0.256
RoboMimic	60,674	0.166	0.388	0.394	0.527	90.9	0.289

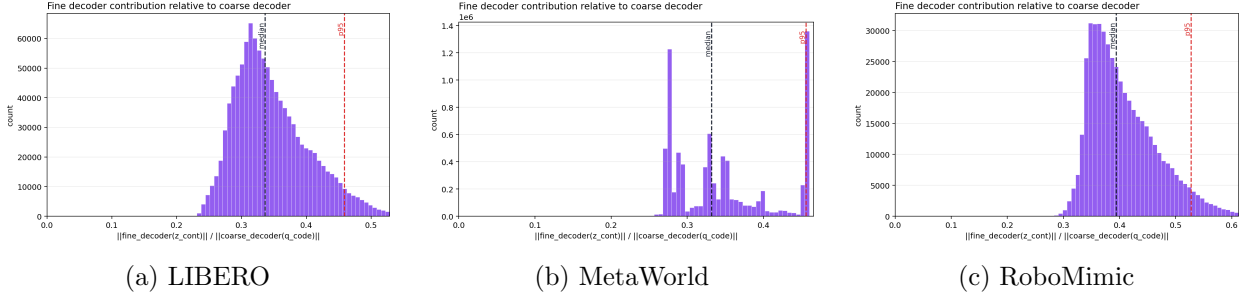


Figure 8: Distribution of the fine-to-coarse decoder contribution ratio $r = \|\text{Dec}_{\text{fine}}\|_2 / \|\text{Dec}_{\text{coarse}}\|_2$ over all events in each benchmark family. The black dashed vertical line marks the median and the red dashed vertical line marks the 95th percentile in each dataset. Values below 1.0 mean that the continuous residual contributes a smaller event embedding norm than the discrete-token branch; values below 0.5 mean that the residual branch contributes less than half of the coarse branch. The mass of all three distributions is well below 1.0, indicating that the residual acts as a secondary metric correction rather than replacing the discrete event token.

Table 7: Dataset-level hard-duration statistics.

Family	Chunks	Range	Std.	MAD	Entropy
LIBERO	138,090	0–15	1.18	0.84	0.978
MetaWorld	1,000,000	1–25	4.02	2.67	0.828
RoboMimic	60,674	1–19	1.58	1.11	0.966

4–6 average only about two frames, while the last event averages 11.4 frames and has the largest cross-chunk standard deviation. This pattern is not consistent with a disguised uniform partition: the tokenizer allocates more event resolution to short transition phases and compresses long stable suffixes into extended events. LIBERO and RoboMimic are closer to uniform on average, but they still show broad duration support, with maximum hard durations of 15 and 19 frames, respectively. These statistics support the central claim that the duration head learns non-uniform temporal structure from action data.

5.8 Ablation Study

We ablate the main components of HEAT using the same LIBERO evaluation protocol as Table 1. Each variant preserves the EventFormer policy and training pipeline except for the named intervention. The uniform-duration variant replaces learned event durations with equal-length events. The no-residual variant sets the continuous event residual to zero, while leaving duration prediction and the action refiner enabled. The token-only policy variant trains the policy with token cross entropy only and uses a token-only decode path at inference, with zero continuous residual, uniform duration, and no action refiner, so that untrained auxiliary heads do not affect control. The no-refiner variant

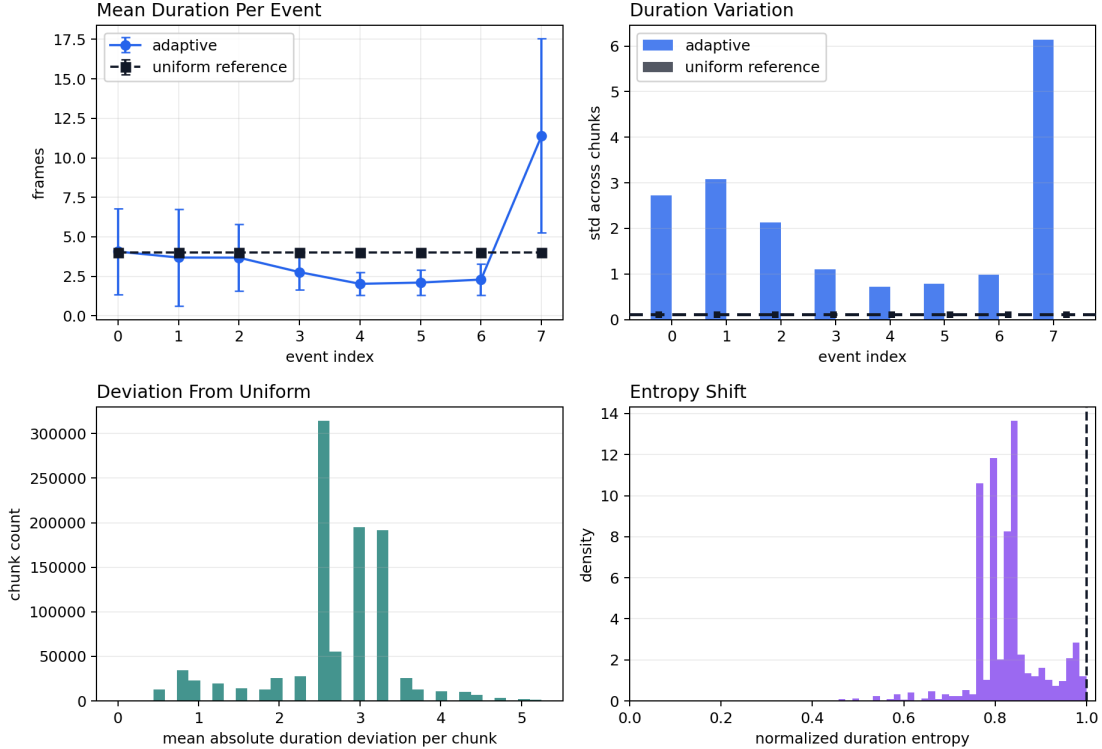


Figure 9: Dataset-level duration distribution on MetaWorld. Top-left: the blue curve shows the mean hard duration at each event index, with error bars indicating standard deviation across chunks; the black dashed line is the four-frame uniform partition. This panel shows where the tokenizer spends or saves temporal extent on average. Top-right: blue bars show the standard deviation of each event duration across chunks, while the black dashed zero line with square markers is the uniform reference, whose duration is fixed and therefore has zero variation. Larger blue bars indicate event positions whose temporal extent changes substantially from chunk to chunk. Bottom-left: each bar counts chunks by their mean absolute duration deviation from the uniform four-frame allocation, so mass away from zero indicates non-uniform timing. Bottom-right: the purple histogram shows normalized duration entropy for individual chunks; the black dashed line at 1.0 is the uniform partition, and a leftward shift means the event durations within a chunk are uneven rather than all equal.

keeps the hybrid event code but executes the tokenizer-decoded actions directly.

Table 8: HEAT ablations on LIBERO.

Variant	Success	Drop
Full HEAT	62.3	–
Uniform duration	52.2	10.1
No continuous residual	60.6	1.7
Token-only policy	42.8	19.5
No action refiner	48.6	13.7

Table 8 and Fig. 10 show that the event representation benefits from both temporal adaptivity and closed-loop metric correction. Replacing learned durations with uniform durations reduces

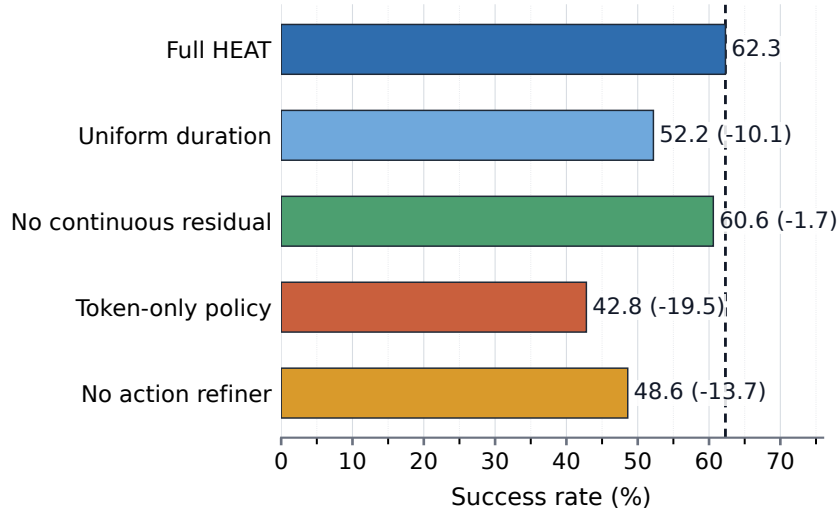


Figure 10: Success rate of HEAT ablations on LIBERO. Parenthesized values denote absolute drop from Full HEAT. The token-only policy and no-action-refiner variants produce the largest degradation, while fixed event timing also causes a clear drop.

success by 10.1 points, indicating that fixed temporal partitions are less effective than learned event boundaries for manipulation phases. Removing the action refiner causes a larger 13.7-point drop, showing that the decoded event plan still benefits from short-horizon observation-conditioned correction. The token-only policy has the largest degradation, falling to 42.8%, which supports the policy objective above: token prediction alone is not sufficient for high-precision continuous control. Removing the continuous residual alone causes only a small drop. This is consistent with the residual analysis in Table 6: the residual branch is useful but subordinate, and part of its role can be compensated by the discrete event token and action refiner.

5.9 HEAT Efficiency and Complexity

We profile the computational cost of HEAT and the discrete or latent action-representation baselines used in the HEAT study. All measurements in this subsection concern the HEAT/EventFormer pipeline and the matched baseline implementations; they do not measure the Qwen backbone or flow-matching action expert used by Continuous-HEAT. The benchmark instantiates each method’s tokenizer together with the autoregressive policy evaluation harness used in the HEAT comparison, but replaces dataset loading, simulation, and heavy vision encoders with synthetic action and state tensors to isolate action-interface cost. The benchmark uses a matched token-budget preset: VQ-BeT uses the same number of residual vector-quantization (VQ) groups as the HEAT event count, $K = 8$, and FAST policy generation uses the measured byte-pair encoding (BPE) padded length rather than the conservative HD_a upper bound. We sweep batch size $B \in \{1, \dots, 2048\}$ and horizon $H \in \{4, 8, 16, 32, 64\}$ with action dimension $D_a = 7$ on CUDA with bfloat16 autocast. For each method, we measure tokenizer training and inference together with policy training and autoregressive inference. Policy inference is especially sensitive to the autoregressive action-token length L , because tokens are generated sequentially, while teacher-forced policy training can parallelize over L but still stores activations and attention tensors whose memory grows with sequence length. In Table 9, times are milliseconds per batch, Peak GB is CUDA memory allocated during policy training, L_{AR}

is the autoregressive generation length, and L_{meas} is the measured tokenizer output length.

Table 9: HEAT action-interface complexity and matched baselines at $B = 1024$, $H = 32$, and $D_a = 7$. Continuous-HEAT is not included.

Method	L_{AR}	L_{meas}	Tok. train	Tok. infer	Pol. train	Pol. infer	Peak GB
Bin	224	224	0.3	0.3	69.2	879.4	14.81
FAST	143	143	1416.4	1699.2	325.4	1909.3	10.52
HEAT	8	8	26.6	11.4	50.3	39.3	1.14
OAT	8	8	25.0	15.8	28.9	28.9	0.77
QueST	8	8	33.2	13.1	30.2	28.8	0.81
VQ-BeT	8	8	26.4	5.9	31.8	29.5	1.42

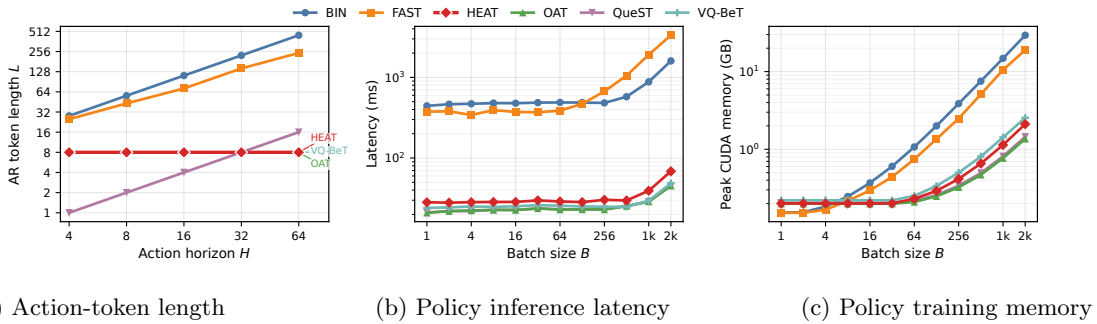


Figure 11: Complexity scaling for HEAT and its matched action-representation baselines; Continuous-HEAT is not included. (a) Autoregressive action-token length as the horizon grows. Binning scales as HD_a , FAST follows its measured BPE length, HEAT, OAT, and VQ-BeT use a fixed eight-token interface, and QueST grows as $H/4$. (b) Policy inference latency at $H = 32$. Long coordinate-level and BPE sequences dominate generation cost, while compact event or latent interfaces remain much flatter. (c) Peak CUDA memory during policy training at $H = 32$. The memory of binning and FAST grows rapidly with batch size because their action-token sequences are long.

Table 9 and Fig. 11 show the main HEAT tradeoff. Coordinate binning is nearly free as a tokenizer, but it exposes a long sequence to the autoregressive policy: at $H = 32$ and $D_a = 7$, the action interface has $L_{\text{AR}} = 224$ tokens. FAST reduces this to 143 measured BPE tokens under the matched preset, but the compression does not make it a short action interface. Its tokenizer first applies trajectory transforms and subword processing, and the local FAST BPE path is CPU-oriented even though the policy benchmark itself runs on CUDA. A fully GPU-native FAST tokenizer would require a separate batched BPE implementation, so we report the cost of the available algorithmic path and use the measured BPE length rather than the conservative HD_a generation bound. The remaining policy cost is still dominated by autoregressive decoding over a long variable-length sequence. At $B = 1024$, HEAT reduces policy inference latency from 879.4 ms for binning and 1909.3 ms for FAST to 39.3 ms, corresponding to $22.4\times$ and $48.6\times$ speedups. The corresponding policy-training memory is 1.14 GB for HEAT, compared with 14.81 GB for binning and 10.52 GB for FAST, a $13.0\times$ and $9.2\times$ reduction. These values characterize the HEAT/EventFormer pipeline and should not be interpreted as latency or memory measurements for Continuous-HEAT.

Among compact learned representations, HEAT is not the cheapest interface. OAT, QueST, and VQ-BeT all use eight action tokens at $H = 32$ in this matched comparison, and their policy inference times are 28.9, 28.8, and 29.5 ms, respectively, compared with 39.3 ms for HEAT. These

methods have simpler decoded targets: OAT and QueST predict compact discrete/register tokens, while VQ-BeT performs fast vector-quantized lookup and offset decoding. HEAT pays extra cost for duration prediction, hybrid residual decoding, and the closed-loop action refiner. With eight VQ groups, VQ-BeT has inference latency comparable to OAT and QueST, while its larger offset head gives a policy-training peak memory of 1.42 GB. HEAT’s additional cost provides a representation that preserves metric action detail and temporal structure, as shown by the HEAT reconstruction, duration, residual, and rollout analyses above. Within this HEAT-specific comparison, HEAT occupies a middle point in the design space: much more efficient than per-coordinate or long BPE action tokens, while more expressive than a purely latent interface with no explicit event duration or residual-refinement structure.

5.10 Qualitative Analysis

Fig. 5 visualizes two representative samples encoded by the frozen tokenizer. The qualitative pattern is consistent across both examples. During the prefix, the end-effector action dimensions change rapidly and the velocity norm contains several local peaks. HEAT responds by allocating short events of one to three frames, placing boundaries around the motion changes. After the object-motion transient, the action becomes nearly constant and the tokenizer merges the stable suffix into much longer events. This behavior contrasts with the gray dashed uniform reference, whose boundaries remain fixed every four frames regardless of whether the sample is changing or holding.

The same examples also provide a simple diagnostic for whether the learned boundaries are merely a disguised uniform partition. For each sample, we compare HEAT boundary frames with the largest action-change frames measured by the norm of finite differences. In the two visualized samples, the mean nearest-boundary distance to these high-change frames is 0.57 frames for HEAT versus 1.43 frames for the uniform partition. Although this statistic is only a local qualitative diagnostic, it supports the intended interpretation of the tokenizer: learned durations concentrate temporal resolution near phase transitions, while stable motion segments are represented compactly by longer events.

6 Discussion

The results support a broader interpretation of event-adaptive action modeling. HEAT separates reusable event identity, temporal extent, and metric correction into a compact code for autoregressive prediction. Continuous-HEAT retains the same temporal abstraction while replacing quantized event content with continuous latents that can condition a flow-matching action generator. Thus event structure is not tied to a particular policy family or decoding mechanism.

Several limitations remain. HEAT and Continuous-HEAT use different event-content parameterizations and should be compared through controlled baselines rather than a single ranking. Continuous-HEAT uses an action-derived encoder as a privileged training teacher; although the teacher is detached from the inference path, its effect should be audited with teacher-free and event-ablation experiments. The two-stage HEAT pipeline also requires a well-trained representation before policy training. Complex long-horizon domains such as RoboCasa remain challenging in the available logs, and real-robot deployment will require validation under sensor noise, dynamics mismatch, and safety constraints.

7 Conclusion

We introduced Hybrid Event-Adaptive Action Representations for Robot Learning, a framework that discovers variable-duration events from continuous action chunks and exposes them through policy-compatible event interfaces. We instantiate the framework as HEAT, a discrete-continuous event representation with FSQ tokens and bounded residuals for autoregressive policies, and Continuous-HEAT, a continuous event-conditioned interface for Qwen3.5 flow-matching VLA models. The HEAT experiments on LIBERO, MetaWorld, and RoboMimic establish the value of event-adaptive timing and hybrid action coding, while the Qwen3.5 experiments show that Continuous-HEAT retains these gains with continuous flow-matching control using a Qwen3.5-0.8B backbone in an approximately 1B-parameter overall model.

References

- [1] Open X-Embodiment Collaboration et al. Open X-Embodiment: Robotic learning datasets and RT-X models. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, 2024.
- [2] Octo Model Team et al. Octo: An open-source generalist robot policy. In *Robotics: Science and Systems*, 2024.
- [3] Moo Jin Kim et al. OpenVLA: An open-source vision-language-action model. In *Proceedings of the Conference on Robot Learning*, pages 2679–2713, 2025.
- [4] Tony Z. Zhao et al. Learning fine-grained bimanual manipulation with low-cost hardware. In *Robotics: Science and Systems*, 2023.
- [5] Cheng Chi et al. Diffusion policy: Visuomotor policy learning via action diffusion. In *Robotics: Science and Systems*, 2023.
- [6] Yaron Lipman et al. Flow matching for generative modeling. In *International Conference on Learning Representations*, 2023.
- [7] Kevin Black et al. π_0 : A vision-language-action flow model for general robot control. In *Robotics: Science and Systems*, 2025.
- [8] Seungjae Lee et al. Behavior generation with latent actions. In *Proceedings of the International Conference on Machine Learning*, pages 26959–26975, 2024.
- [9] Atharva Mete et al. QueST: Self-supervised skill abstractions for learning continuous control. In *Advances in Neural Information Processing Systems*, volume 37, pages 4062–4089, 2024.
- [10] Karl Pertsch et al. FAST: Efficient action tokenization for vision-language-action models, 2025.
- [11] Chaoqi Liu et al. OAT: Ordered action tokenization, 2026.
- [12] Fabian Mentzer et al. Finite scalar quantization: VQ-VAE made simple. In *International Conference on Learning Representations*, 2024.
- [13] Ashish Vaswani et al. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017.
- [14] Suraj Nair et al. R3m: A universal visual representation for robot manipulation, 2022.
- [15] Ted Xiao et al. Masked visual pre-training for motor control, 2022.
- [16] Oren Rippel, Michael A. Gelbart, and Ryan P. Adams. Learning ordered representations with nested dropout. In *Proceedings of the International Conference on Machine Learning*, pages 1746–1754, 2014.
- [17] Aaron van den Oord, Oriol Vinyals, and Koray Kavukcuoglu. Neural discrete representation learning. In *Advances in Neural Information Processing Systems*, 2017.
- [18] Karl Pertsch, Youngwoon Lee, and Joseph Lim. Accelerating reinforcement learning with learned skill priors, 2020.

- [19] Pierre-Luc Bacon, Jean Harb, and Doina Precup. The option-critic architecture, 2017.
- [20] Ofir Nachum, Shixiang Gu, Honglak Lee, and Sergey Levine. Data-efficient hierarchical reinforcement learning, 2018.
- [21] Ajay Mandlekar et al. What matters in learning from offline human demonstrations for robot manipulation. In *Proceedings of the Conference on Robot Learning*, 2021.
- [22] Alec Radford et al. Learning transferable visual models from natural language supervision. In *Proceedings of the International Conference on Machine Learning*, 2021.
- [23] An Yang et al. Qwen3 technical report, 2025.
- [24] Qwen Team. Qwen-VLA: Unifying vision-language-action modeling across tasks, environments, and robot embodiments, 2026.
- [25] Bo Liu et al. LIBERO: Benchmarking knowledge transfer for lifelong robot learning. In *Advances in Neural Information Processing Systems*, 2023.
- [26] Tianhe Yu et al. Meta-World: A benchmark and evaluation for multi-task and meta reinforcement learning. In *Proceedings of the Conference on Robot Learning*, pages 1094–1100, 2020.
- [27] Soroush Nasiriany et al. RoboCasa: Large-scale simulation of everyday tasks for generalist robots. In *Robotics: Science and Systems*, 2024.
- [28] Yuke Zhu et al. robosuite: A modular simulation framework and benchmark for robot learning, 2020.
- [29] Oier Mees et al. Calvin: A benchmark for language-conditioned policy learning for long-horizon robot manipulation tasks, 2022.
- [30] Alexander Khazatsky et al. Droid: A large-scale in-the-wild robot manipulation dataset, 2024.
- [31] Homer Walke et al. Bridgedata v2: A dataset for robot learning at scale, 2023.